# Documentation du programme

## Introduction

Cette documentation doit permettre de comprendre et de réutiliser le code python associé au modèle MEP développé par Karine Watrin lors d'un stage avec l'aide de Didier Paillard, à partir des codes de Didier Paillard, Correntin Herbet, Vincent Labarre.

Avant de présenter plus en détail les aspects plus techniques, une première partie rappelle la théorie physique du modèle, et donne également quelques bases d'optimisation pour la résolution du problème associé au cycle de l'eau.

Puis vient la documentation du code, divisée en version. Ce programme a en effet été conçu pour être utilisé comme une boîte noire, tout en ayant une structure permet des ajouts fonctionnels.

## Table des matières


Introduction ..... 1

I. Présentation du modèle MEP de convection atmosphérique ..... 4

Le modèle physique simple de l'atmosphère en 1D ..... 4

Structure du modèle ..... 4

Définition de l'énergie ..... 5

Définition des flux ..... 5

Schéma ..... 6

Bilan énergétique ..... 6

Déductions possibles de ce modèle ..... 7

Le modèle MEP de Didier Paillard et Vincent sans le cycle de l'eau ..... 7

Hypothèse de la saturation de la vapeur d'eau ..... 7

Hypothèse de la maximisation de la production d'entropie ..... 7

Résumé de la démarche ..... 8

Le modèle MEP avec le cycle de l'eau ..... 8

Intérêt de l'intégration du cycle de l'eau et remarques préliminaires ..... 8

Intégration des précipitations ..... 8

Schéma ..... 9

Modèle avec le cycle de l'eau à saturation ..... 9

Modèle avec le cycle de l'eau sans saturation ..... 10

II. Problème d'optimisation sous contrainte ..... 11

Transformation des problèmes d'optimisation ..... 11

Quelques notions d'optimisation et nomenclature ..... 11

Transformation de problème au travers du choix de variable ..... 11
Expression de l'entropie et des contraintes en fonction des choix de variables d'optimisation pour tous les modèles..... 12
Choix possibles pour le modèle MEP sans cycle de l'eau:..... 12
Choix possibles pour le modèle MEP avec cycle de l'eau à saturation: ..... 13
Remarques sur les intervalles de validité de la masse et de la température..... 14
III. Méthode de résolution numérique et implémentation ..... 15
Optimisation numérique avec *scipy.optimize.minize* ..... 15
Choix de la méthode d'optimisation..... 15
Paramètres d'entrée de la fonction *minimize* ..... 15
Récapitulatif de l'algorithme ..... 16
La linéarisation des flux radiatifs ..... 16
Méthode de résolution ..... 17
Méthode simple..... 17
Méthode convergence ..... 17
Méthode threshold ..... 17
Méthode maxiter variation ..... 17
Méthode value variation..... 17
Dernières améliorations pour une convergence plus robuste ..... 17
Les changements de variables..... 18
Test de 'feasibility' ..... 18
La résolution en 2 étapes ..... 19
Récapitulatif des choix de paramètres possibles pour le calcul numérique et de l'algorithme complet . 19
Choix possibles des paramètres..... 19
Récapitulatif de l'algorithme complet..... 20
IV. Le programme python et les différentes versions ..... 21
Généralité sur le programme python : ..... 21
La nomenclature ..... 21
Les différents fichiers du programme ..... 21
Les 3 différentes versions du code..... 22
La version simple ..... 24
Le choix des paramètres avec main.py ..... 24
L'ajout d'un modèle ou d'une contrainte..... 25
La version complète..... 26
Le choix des paramètres avec main.py ..... 26
La version exhaustive..... 28
Le choix des paramètres avec main.py ..... 28

# I. Présentation du modèle MEP de convection atmosphérique

Afin de mieux comprendre les flux atmosphériques, les chercheurs établissent des modèles physiques théorique et numérique. Dans le cas qui nous intéresse, l'objectif est de trouver un modèle qui permette, à partir d'un nombre de données géographiques et chimiques restreints, d'établir le profil de température, d'énergie et de flux dans l'atmosphère à différentes latitudes.

Ces modèles de climat reposent sur un ensemble d'équations physiques, souvent insuffisantes pour déterminer toutes les variables, ou bien impossibles à résoudre pour les échelles spatiales considérées. C'est pourquoi il est nécessaire de poser des hypothèses simplificatrices, ou hypothèse de « fermeture », afin d'obtenir un problème solvable numériquement.

Une hypothèse de fermeture possible est la maximisation de la production de l'entropie (MEP), qui stipule qu'un système ouvert maximise sa production d'entropie. Si cette hypothèse n'est pas prouvée théoriquement, elle a néanmoins déjà permis d'obtenir des résultats pertinents : les résultats du modèle de flux atmosphériques de Paltridge (1975) reposant sur cette hypothèse sont très proches des observations empiriques.

Pour approfondir la recherche sur l'hypothèse MEP, Didier Paillard et Vincent Labarre ont développé un modèle de convection atmosphérique reposant sur cette hypothèse. Ce modèle, un peu plus poussé que le modèle de Paltridge, est plus précis sur les différentes formes d'énergies en jeu. Cet ajout présente néanmoins quelques simplifications, notamment des hypothèses assez arbitraires sur la présence d'eau. C'est pourquoi revoir ces hypothèses sur la présence de l'eau permettrait a priori d'améliorer le modèle, et dans cette optique, un modèle prenant en compte le cycle de l'eau a été développé par Didier Paillard. Ces deux modèles sont décrits plus précisément par la suite, après la présentation du modèle physique qui a servi de base à ces modèles.

## Le modèle physique simple de l'atmosphère en 1D

Le modèle décrit par la suite est relativement simple et a servi de base pour le développement du modèle MEP. Ce modèle sert doit permettre d'obtenir, à partir de données empiriques simples et connues, les profils de températures, d'énergies et de flux convectifs de l'atmosphère.

### Structure du modèle

L'atmosphère est modélisée très simplement, en une dimension, par une colonne de $n$ couches (ou boîtes), avec une couche supplémentaire à sa base (la couche *0* qui correspond à la couche limite).

Chaque couche *i* est caractérisée par

- sa température $T_i$ (en Kelvin, donc forcément positivie)
- son énergie spécifique $e_i$,
- son humidité absolue $q_i$ (c'est-à-dire la masse d'eau sous forme de vapeur par masse d'air sec) – équivalente à son humidité relative $h_i$ (qui est le rapport entre l'humidité absolue et l'humidité maximale (à saturation) obtenue dans les mêmes conditions de température et de pression),
- sa pression $p_i$,
- sa teneur en $CO_2$ et $O_3$.

La couche *0* correspond à la couche limite, c'est-à-dire la couche d'air en équilibre avec la surface de la planète. Cette surface peut être le sol terrestre, comme l'océan, et elle est caractérisée par un albedo $\alpha$ et un niveau d'humidité. Ainsi, l'humidité de la couche limite sera maximale si celle-ci est au contact de l'océan

ou d'un sol très humide (climat tropical). La $n^{ième}$ couche correspond à la couche la plus élevée de l'atmosphère.

## Définition de l'énergie

L'énergie spécifique (ou énergie massique) est définie comme la somme de l'énergie thermique de l'air sec, l'énergie géo potentielle et l'énergie latente de l'eau présente sous forme de vapeur. Plus précisément, l'énergie spécifique de la couche *i* est définie selon la formule (1) de l'article de Vincent :

$$e_i = Cp * T_i + g * z_i + L * q_i$$

Un ensemble d'hypothèse détaillées dans l'article de Vincent ici ont été faites afin de transformer légèrement cette expression et ne la faire dépendre que de deux inconnues :

$$e_i = f(T_i, q_i)$$

La définition de l'énergie donnée ici est la définition la plus complète, celle de l'énergie dite '*moist'*.
L'article de Vincent définie également deux autres énergies, qui peuvent être utiles : '*dry'* (sans le terme d'énergie latente de la vapeur d'eau) et *'cpT'* (uniquement l'énergie potentielle).

## Définition des flux

Les échange d'énergies et de matières entre les différentes couches de l'atmosphère et les couches limites sont modélisés par des flux convectifs $F$ et des flux radiatifs $R$.

### *Flux convectifs*

Les flux convectifs sont plus précisément définis ainsi : entre deux couches juxtaposées, un flux convectif va de la couche inférieur *i* à la couche supérieur *i+1*, transportant ainsi une énergie $m * e_i$ vers la couche supérieure *i+1* (où $m$ est le flux massique et $e_i$ l'énergie spécifique de la couche *i*). Et inversement, un flux convectif va de la couche *i+1* vers la couche *i* et apporte ainsi une énergie $m * e_{i+1}$ à la couche *i*. Les flux massiques $m$ sont choisis égaux, il n'y a donc pas de variation de masse, ce qui permet ainsi de définir des couches dont la masse, et par conséquent la pression, sont des constantes. Le flux entre la couche *i* et la couche *i+1* est donc défini selon la formule (3) de l'article de Vincent (Attention, il y a un changement d'indice pour $F$ ) :

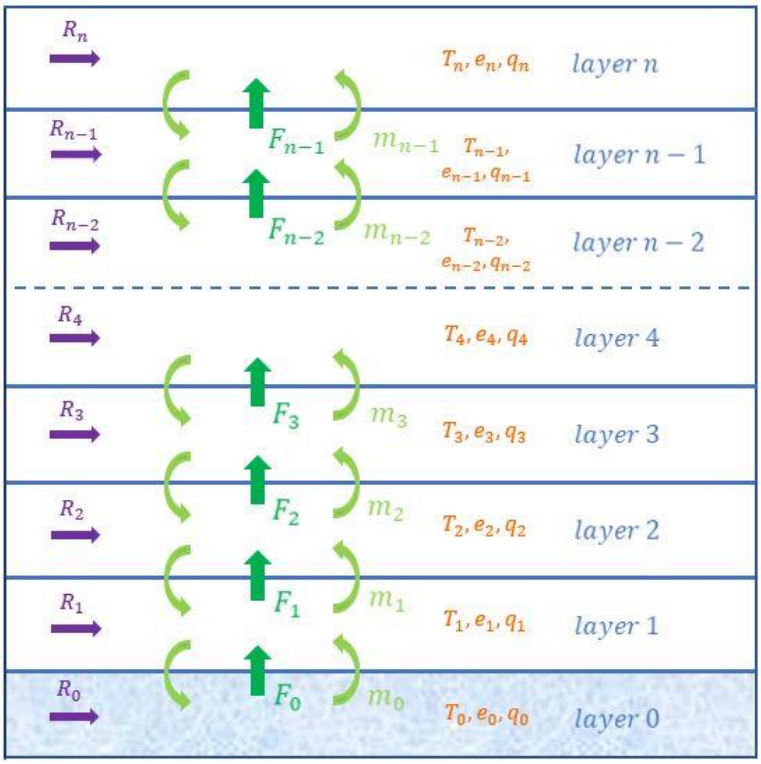
$$F_i = m_i * (e_i - e_{i+1})$$

Le coefficient $m$ peut être interprété comme un flux massique, mais également comme un coefficient de mélange. Dans tous les cas il ne peut être que positif ou nul.
Il n'y a pas de flux convectifs entre l'extérieur et l'intérieur de la colonne.

### *Flux radiatifs*

Les flux radiatifs sont eux calculés selon l'article de Corentin Herbet, à partir de profils de concentration fixés. Ils peuvent être déterminés à partir de profils de concentrations en $CO_2$ et $O_3$, de l'albédo α du sol et du profils de température.

## Schéma




Ainsi, il y a

- $n+1$ valeurs pour les profils de $q, T, E$, et $R$
- $n$ valeurs pour $F$ et $m$

Remarque sur les conventions d'indice :
Pour rester cohérent avec le formalisme de python, tous les indices commencent à 0 et les variables toujours égales à 0 ne sont pas considérées. Il y a donc quelques changements dans la notation par rapport à l'article de Vincent, et quelques décalages entre les indices des grandeurs et les indices des boîtes. Par exemple, il y a $n$ valeurs pour $F$ et $m$, et non $n+2$ comme dans l'article de Vincent. Ces changements ne changent rien à la théorie ni aux résultats.

## Bilan énergétique

### *Bilan énergétique pour une boîte*

A l'équilibre, les flux et les grandeurs physiques caractéristiques d'une couche sont constantes. Si l'énergie d'une couche est constante dans le temps, alors les flux énergétiques entrant et sortant sont égaux. Ainsi, selon la formule (4) de l'article de Vincent :

$$R_i + F_{i-1} - F_i = 0$$

avec comme cas particulier $R_0 - F_0 = 0$ et $R_n + F_{n-1} = 0$

### *Bilan énergétique sur toute la colonne*

A l'équilibre, la somme des flux radiatif $R_i$ doit donc être nulle, ou ce qui se traduit par la nullité du flux radiatif entre la dernière couche et l'espace

## Déductions possibles de ce modèle

Les différentes équations établies permettent de montrer les liens entre le profil d'énergie, les flux convectifs et les flux radiatifs. Si l'on considère que les profils de concentrations en $CO_2$ et $O_3$, le profil de pression $p$ et l'albédo α sont donnés, alors le profil d'énergie $E$, les flux convectifs $F$ et les flux radiatifs $R$ ne sont finalement dépendants que de deux variables. Ces variables sont le profil de température $T$ et le profil d'humidité $q$ (ou $h$). Ce sont donc les deux inconnues du problème, et il faut encore ajouter des hypothèses afin de les déterminer leurs valeurs.

Plus précisément :

- les données sont les profils de concentrations en $CO_2$ et $O_3$, le profil de pression $p$ et l'albédo α
- les variables qui sont intéressantes à déterminer sont $q, T, E, F, m$ et $R$, soit $6n + 4$ inconnues
- les équations permettant de déterminer ces inconnues sont au nombre de , ainsi :
  - $n + 1$ équations relient $T$ et $R$ par la définition des flux radiatifs
  - $n + 1$ équations relient $T$ et $E$ par la définition de l'énergie
  - $n$ équations relient $F$ et $m$ par la définition des flux radiatifs
  - $n + 1$ équations relient $F$ et $R$ par le bilan énergique

Il manque donc $2n + 1$ équations afin de pouvoir relier toutes les variables et de déduire des données en entrées les grandeurs physiques qui nous intéressent.

# Le modèle MEP de Didier Paillard et Vincent sans le cycle de l'eau

Afin de déterminer $E, F$ et $R$, il est nécessaire de poser des hypothèses supplémentaires.

## Hypothèse de la saturation de la vapeur d'eau

Pour rappel, lorsque de l'eau est présente dans l'air atmosphérique, elle peut l'être sous forme liquide, solide et gazeux à la fois. La quantité d'eau présente sous forme gazeuse (c'est-à-dire l'humidité absolue $q$) est limitée : elle peut varier en fonction de la température et de la pression, mais, pour une température $T_d$ et une pression $p_d$ , elle ne peut dépasser une certaine valeur, $q_{saturation}(T_d, P_d)$. Toute l'eau restante est alors sous forme liquide ou solide ; nous ne considérerons que la forme liquide dans le cadre de ce modèle pour simplifier.

La première hypothèse supplémentaire est celle de considérer que l'air est toujours saturé en vapeur d'eau, et que l'eau sous forme liquide et la conservation des flux massiques ne sont pas pris en compte. En d'autres termes, il n'y a pas de précipitations, et la quantité de vapeur d'eau qui peut se former n'est pas limitée par la quantité d'eau liquide présente dans l'air.

Avec cette hypothèse, le profil d'humidité devient une fonction de la température. En effet, il existe une formule empirique qui, à partir de $T$ et de $p$ permet d'obtenir la valeur $q_{saturation}$ et donc par égalité, $q$. Cette hypothèse permet ainsi d'ajouter $n + 1$ équation. La seule inconnue restante est donc $T$, et il faut encore une hypothèse de « fermeture » pour pouvoir résoudre le système d'équation.

## Hypothèse de la maximisation de la production d'entropie

La deuxième hypothèse posée est celle de la maximisation de la production d'entropie (MEP), présentée dans l'introduction.

L'entropie est définie ainsi :

$$\sigma(T) = \sum_{i=0}^{i=n} \frac{F_{i-1} - F_i}{T_i} \quad = \sum_{i=0}^{i=n} \frac{-R_i}{T_i}$$

Ainsi, cette hypothèse indique que les valeurs correctes de $T$ sont celles qui maximisent $\sigma$. Pour trouver ces valeurs de $T$ il faut donc résoudre le problème d'optimisation suivant : maximisez $\sigma(T)$.

Il y a néanmoins quelques contraintes à ajouter à ce problème d'optimisation pour que la solution du problème d'optimisation soit cohérente avec la solution du problème physique : il faut que $m$ calculé à partir des valeurs de $T$ trouvées soit positive.

### Résumé de la démarche

La démarche pour obtenir les profils d'énergie, de flux et de température est donc la suivante : les valeurs de $T$ sont obtenues en résolvant le problème d'optimisation et à partir de ces valeurs de $T$ il est possible de calculer les autres profils grâces aux équations obtenues plus hauts.

## Le modèle MEP avec le cycle de l'eau

### Intérêt de l'intégration du cycle de l'eau et remarques préliminaires

Le modèle sans le cycle de l'eau est relativement simple, et donc plus facilement solvable numériquement. Néanmoins la présence de l'eau sous forme liquide et vapeur est très simplifiée. Une possibilité d'amélioration du modèle est donc d'y d'intégrer la présence d'eau sous forme liquide, et de modéliser ainsi plus précisément le cycle de l'eau.

Dans la suite de cette partie, nous considérons donc le modèle simple de l'atmosphère en 1D, auquel il manque $2n + 1$ équations pour être solvable. La définition de l'énergie est celle de *'moist'*.

Pour rappel, l'eau liquide peut être en apesanteur dans l'air, comme elle peut également précipiter. A priori l'eau liquide précipite si son énergie est insuffisante pour rester en apesanteur. Par la suite, l'eau liquide en apesanteur ne sera pas étudiée, pour simplifier le problème. Nous faisons donc l'hypothèse que sa quantité influe peu les autres grandeurs physiques comparé aux précipitations.

Le cycle de l'eau modélisé se résume donc ainsi : l'eau ne peut gagner de l'altitude que sous forme de vapeur grâce aux flux convectifs, et l'eau ne peut perdre de l'altitude que sous forme de précipitation, et la quantité d'eau sur l'ensemble de la colonne est conservée.

### Intégration des précipitations

Les précipitations sont intégrées au modèle grâce au principe de conservation de la masse d'eau.

#### *Bilan local de conservation de la masse*

Si la quantité de vapeur d'eau apporté par un flux de la couche i vers la couche i+1 est plus grande que la quantité de vapeur possible (limitée par $q_{saturation}$), alors la différence se liquéfie et précipite.

Pour simplifier, nous considérerons qu'en précipitant, l'eau liquide traverse les autres couches inférieures mais n'influe aucunes de leurs caractéristiques. Les précipitations rejoignent donc toutes la couche *0* quelles que soient leurs boîtes d'origine. Avec ce modèle de précipitation, la quantité d'eau dans chaque boîte ne peut que diminuer avec l'altitude (ce qui n'est pas le cas dans le modèle précédent).

Les précipitations, symbolisées par la lettre $L$ (pour *liquid*) sont donc définie ainsi:

$$L_i = m_i(q_i - q_{i+1}) - m_{i+1}(q_{i+1} - q_{i+2})$$

$$\text{avec } L_i > 0$$

La contrainte de positivité de $L$ s'explique physiquement : l'eau ne peut que précipiter vers le bas, elle ne peut donc pas rejoindre des couches supérieurs (du moins avec les choix de modélisation effectués).

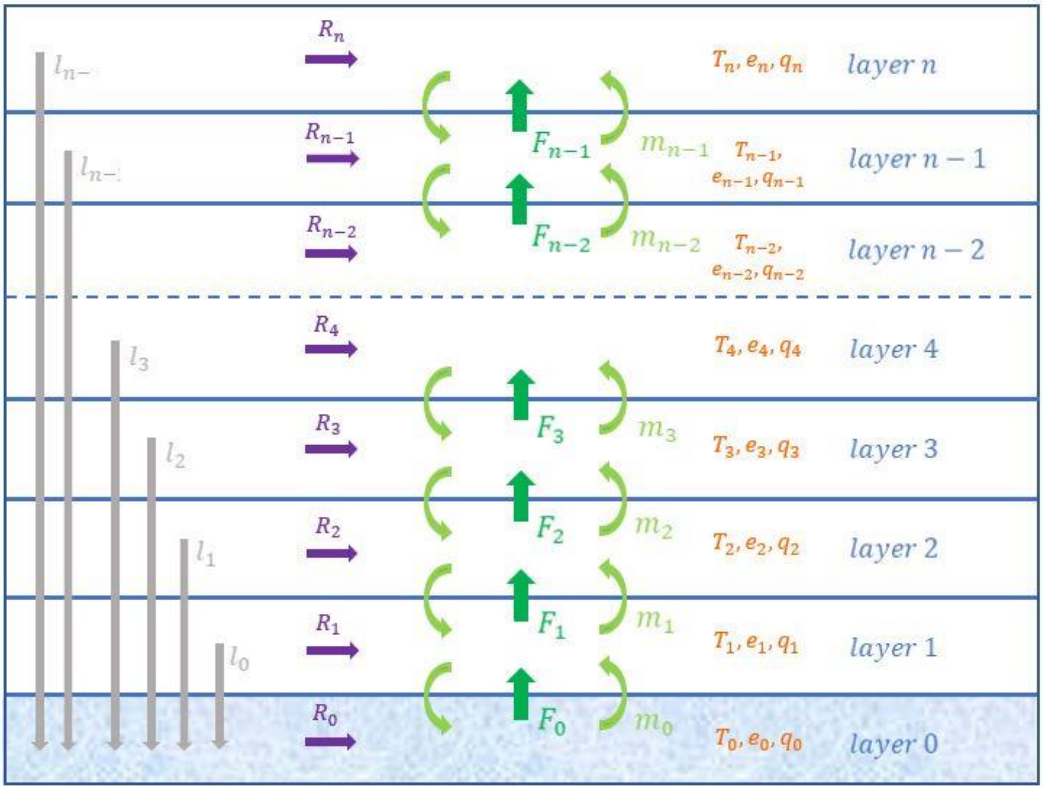
### *Bilan de conservation de la masse à l'interface avec la couche limite*

A l'état d'équilibre, tous les flux d'eau doivent s'équilibrer. En particulier, à la couche limite, toutes les précipitations arrivant d'une altitude plus élevée doivent être compensées par le flux convectif qui va vers la couche *1*. Ainsi, une deuxième contrainte est explicitée :

$$m_0(q_1 - q_0) + \sum_{i=0}^{i=n-1} L_i \ = 0$$

La définition des précipitations impose donc deux contraintes de plus, la positivité de $L$ et l'égalité des flux à l'interface avec la couche *0*. Ces contraintes influencent les valeurs de $q$, en les bornant, mais ne permet pas de les déterminer. Il faut donc encore une hypothèse ou un choix de plus pour déterminer $q$. Deux choix possibles sont examinés par la suite, donnant ainsi deux nouveaux modèles : le cycle de l'eau à saturation, et le cycle de l'eau sans saturation

## Schéma




## Modèle avec le cycle de l'eau à saturation

L'hypothèse déjà faite dans le modèle précédent est reprise : la quantité de vapeur d'eau sera égale à celle lorsqu'il y a saturation de l'air (cf partie précédente).

Comme précédemment, une dernière hypothèse est encore nécessaire pour pouvoir déterminer $T$, et cette hypothèse est logiquement celle de la maximisation de l'entropie. Pour obtenir $T$, il faut donc maximiser l'entropie, avec cette fois-ci 3 contraintes : positivité de $m$, positivité de $L$, et égalité des flux massiques à l'interface avec la couche 0.

## Modèle avec le cycle de l'eau sans saturation

Dans ce modèle, aucune hypothèse sur la saturation de l'air n'est faite. Pour déterminer $T$ et $q$, il est fait le choix d'utiliser l'hypothèse MEP. Cela implique donc que le problème d'optimisation est à priori plus difficile à résoudre car il y a deux fois plus de variables à optimiser. Cependant, les contraintes ne changent pas.

# II. Problème d'optimisation sous contrainte

Les modèles MEP présentés précédemment font donc appel à de l'optimisation sous contrainte. Ces problèmes peuvent être relativement difficiles à résoudre, de par leur grand nombre de variables et la complexité des fonctions et des contraintes ; une réflexion a donc été menée pour transformer au mieux les problèmes d'optimisation sous contrainte pour en faciliter la résolution.

## Transformation des problèmes d'optimisation

### Quelques notions d'optimisation et nomenclature

La fonction à optimiser est appelée la fonction objectif (traduction de l'anglais, donc peut-être impropre ?). Une variable d'optimisation est la variable de la fonction objectif, et la valeur optimal de la variable d'optimisation est l'antécédent de l'optimal de la fonction à optimiser.

Un problème est dit faisable s'il existe au moins un point (c'est-à-dire une valeur de la variable d'optimisation) qui respecte les contraintes et qui appartienne à l'ensemble de définition de la fonction objectif.

Deux problèmes d'optimisation sont dits 'dual' si toute solution de l'un est solution de l'autre. Ce sont finalement des problèmes équivalent mathématiquement. L'intérêt de cette notion de dualité, c'est qu'un problème complexe peut être résolu si un problème 'dual' plus simple à résoudre est trouvé. A priori, plus la fonction objectif et les contraintes seront proches de fonctions convexes, plus la résolution sera simple et rapide numériquement (la théorie est bien plus complexe en réalité).

## Transformation de problème au travers du choix de variable

Les problèmes d'optimisation posés pour trouver une solution au modèle sont relativement complexes : la fonction objectif et les contraintes ne sont pas toujours convexes (ou si c'est le cas, ce n'est pas facile à prouver). Il y a donc un intérêt à chercher une forme plus simple du problème, par exemple en effectuant un changement de variable, en choisissant une variable d'optimisation différente, ou en simplifiant l'expression d'une contrainte

### *Changement de variable pour la simplification des fonctions*

Dans le cas des deux modèles MEP, un changement de variable est particulièrement intéressant

$$T \rightarrow x \ : \ x = \frac{T}{T_{ref}} \qquad \text{où} \ \ T_{ref} = 342K$$

Ce changement de variables permet de simplifier le calcul de la fonction d'entropie, car son expression contient $\frac{1}{T}$

### *Choix de la variable d'optimisation*

Les problèmes MEP peuvent être exprimés selon plusieurs variables. En fonction du choix de variables, l'expression de l'entropie et les contraintes ne sont pas exactement les mêmes. Voici quelques choix qui ont été étudiés :

- pour le modèle sans cycle de l'eau : $F$, $x$, $x\&m$, $x\&m\&F$
- pour le modèle avec le cycle de l'eau sans saturation : $x\&m$, $x\&m\&F$
- pour le modèle avec le cycle de l'eau sans saturation : $x\&m\&h$

Le choix de ou des variables d'optimisation a des conséquences sur l'expression des contraintes ; l'ensemble des choix de variable en fonction des modèles, avec les contraintes, est décrit dans un paragraphe suivant.

### Simplification de la contrainte

Une autre possibilité pour transformer le problème d'optimisation est de trouver une expression plus simple (mais correcte) de la contrainte.

Cette méthode a pu être appliquée une fois, pour l'expression de la contrainte $m(F) > 0$ ou $m\ (x) > 0$ :

$$m\ (x) > 0 \quad \text{où} \quad m_i = \frac{F_i(x)}{e_i(x) - e_{i+1}(x)}$$

équivaut à la contrainte

$$alpha(x) > 0 \quad \text{où} \quad alpha_i = F_i(x) * (e_i(x) - e_{i+1}(x))$$

## Expression de l'entropie et des contraintes en fonction des choix de variables d'optimisation pour tous les modèles

Avec l'ensemble de ces possibilités de transformations de problème, il a été possible de programmer la résolution avec différents choix de variables et différents choix de contraintes dans certains cas. Voici les différents choix de variables (avec l'entropie et les contraintes adaptées) qui ont été étudiés et laissés dans le code final.

### Choix possibles pour le modèle MEP sans cycle de l'eau:

#### $F$

Entropie $\sigma(F) = \sum_{i=0}^{i=n} x(F_i) * (F_{i-1} - F_i)$

Contraintes positivité de $x$: $x(F) > 0$

positivité de $m$ : $m\ (F) > 0$ où $m_i = \frac{F_i}{e_i(F) - e_{i+1}(F)}$

#### $x$

Entropie $\sigma(x) = \sum_{i=0}^{i=n} - R_i(x) * x_i$

Contraintes positivité de $x$: $x > 0$

positivité de $m$ : $m\,(x) > 0$ où $m_i = \frac{F_i(x)}{e_i(x) - e_{i+1}(x)}$

bilan énergétique local : $R_i(x) + F_{i-1}(x) - F_i(x) = 0$

*$x$ et $m$*

Entropie $\sigma(x) = \sum_{i=0}^{i=n} - R_i(x) * x_i$

ou $\sigma(x,m) = \sum_{i=0}^{i=n} x_i * (F_{i-1} - F_i)$ où $F_i = m_i(e_i(x) - e_{i+1}(x))$

Contraintes positivité de $x$: $x > 0$

positivité de $m$ : $m > 0$

bilan énergétique local : $R_i(x) + F_{i-1}(x,m) - F_i(x,m) = 0$

*$x, m$ et $F$*

Entropie $\sigma(F) = \sum_{i=0}^{i=n} x_i * (F_{i-1} - F_i)$

Contraintes positivité de $x$: $x > 0$

positivité de $m$ : $m > 0$

bilan énergétique local : $R_i(x) + F_{i-1} - F_i = 0$

définition de $F$ : $F_i = m_i(e_i(x) - e_{i+1}(x))$

## Choix possibles pour le modèle MEP avec cycle de l'eau à saturation:

*$x$ et $m$*

Entropie : $\sigma(x) = \sum_{i=0}^{i=n} - R_i(x) * x_i$

ou $\sigma(x,m) = \sum_{i=0}^{i=n} x_i * (F_{i-1} - F_i)$ où $F_i = m_i(e_i(x) - e_{i+1}(x))$

Contrainte : positivité de $x$: $x > 0$

positivité de $m$ : $m > 0$

bilan énergétique local : $R_i(x) + F_{i-1}(x,m) - F_i(x,m) = 0$

où $F_i = m_i(e_{i-1} - e_i)$

positivité de $l$ : $l(x,m) > 0$

bilan de masse de l'eau : $m_0(q_1(x) - q_0(x)) + \sum_{i=0}^{i=n-1} l_i\,(x,m) = 0$

*$x, m$ et $F$*

Entropie : $\sigma(F) = \sum_{i=0}^{i=n} x_i * (F_{i-1} - F_i)$

Contrainte : positivité de $x$: $x > 0$

positivité de $m$: $m > 0$

bilan énergétique local : $R_i(x) + F_{i-1} - F_i = 0$

définition de $F$: $F_i = m_i(e_{i-1}(x) - e_i(x))$

positivité de $l$ : $l(x,m) > 0$

bilan de masse de l'eau : $m_0(q_1(x) - q_0(x)) + \sum_{i=0}^{i=n-1} l_i\,(x,m) \;\; = 0$

## Remarques sur les intervalles de validité de la masse et de la température

### *La température*

La contrainte de positivité de $x$ est en réalité plus complexe. Bien sûr, la température étant en Kelvin,

$$T \; > 0 \text{ implique } x \; > 0$$

Néanmoins, la température ne peut pas tendre au-delà des 600 K, et dans l'atmosphère la température ne descend pas non plus vers en dessous de 150K a priori. La contrainte de positivité de $x$ devrait donc plutôt être une contrainte sur des bornes inférieure et supérieur, et lorsque cela était suffisamment simple (ie lorsque $x$ fait partie des variables d'optimisation), la contrainte sur la température est devenue

$$0.5 < x < 2$$

### *La masse*

Comme pour la température $x$, la masse $m$ doit également être bornée pour rester dans un intervalle physiquement réaliste. Le choix de la borne supérieure, quand il est possible, est à définir en entrée de l'algorithme de résolution.

# III. Méthode de résolution numérique et implémentation

La complexité des modèles physiques impose une résolution numérique, qui a été programmée en python ici. Il s'agit maintenant de présenter les choix nécessaires à la résolution numérique et les contraintes liées à l'implémentation des algorithmes en langage python.

## Optimisation numérique avec *scipy.optimize.minize*

Une des raisons pour laquelle le langage python a été choisi est l'existence d'une bibliothèque d'optimisation, *scipy.optimize*. Plus précisément, la fonction *scipy.optimize.minimize* résout efficacement des problèmes d'optimisation sous contraintes selon plusieurs méthodes.

Il faut remarquer que scipy.optimize ne possède pas de fonction *maximize* équivalent à *minimize*, il faut donc inverser la fonction objective, puisque que les modèles MEP sont à maximisation (et non à minimisation) d'entropie.

## Choix de la méthode d'optimisation

La résolution numérique d'un problème d'optimisation sous contrainte se fait de la manière suivante. Tout d'abord, le problème est transformé en problème 'dual' de Lagrange, qui est un problème d'optimisation sans contrainte. L'optimal du problème initial est donc le minimal du Lagrangien. Les méthodes pour trouver ce minimum varient en fonction de la forme du problème, dans les cas les plus complexes ces méthodes s'apparentent souvent à une descente de gradient et nécessitent un nombre important d'itérations. Puis, lorsqu'une solution suffisamment proche du minimum exact est trouvée, la solution du problème initial est calculée à partir de la solution du problème dual.

La fonction *minimize* propose plusieurs choix de méthodes, bien plus complexes que ce qui vient d'être décrit. Celles qui sont pertinentes dans le cas des modèles MEP sont :

- la méthode *'SLSQP'* pour Sequential Linear SQuare Programming
- la méthode *'trust-constraint'*

Pour plus d'information :
https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html

A noté que la méthode '*trust-constraint*' a finalement eu peu de résultats.

## Paramètres d'entrée de la fonction *minimize*

La fonction *minimize* prend en entrée :

- la fonction objectif,
- une valeur initiale arbitraire de la variable d'optimisation,
- la méthode d'optimisation choisie,
- les contraintes,
- [optionnel] les matrices jacobiennes et hessienne de la fonction objectif,
- [optionnel] les matrices jacobiennes et hessienne des contraintes,
- [optionnel] les espace de définition de la variable d'optimisation,
- [optionnel] le nombre maximal d'itération – ou 'maxiter'

Remarques :

- Sur les matrices jacobiennes et hessiennes : leur ajout accélère la convergence très nettement (voir la rend possible dans certains cas) ; par contre, lorsque ces fonctions ne sont pas linéaires, et que l'écart entre la fonction exacte et la fonction approchée par approximation de Taylor, les résultats ne sont pas aussi précis, voir différents entre ceux où la matrice jacobienne est une entrée de *minimize*.
- Sur le maxiter : par défaut, sa valeur vaut 1000. C'est-à-dire que si la fonction *minimize* ne converge pas, elle s'arrête au bout de 1000 itérations et renvoie le dernier résultat qu'elle a obtenu. Le choix de ce paramètre est relativement important, et sera détaillé dans la partie sur la linéarisation des flux radiatifs.

## Récapitulatif de l'algorithme

Pour reprendre la démarche, il faut donc choisir le modèle, les variables d'optimisation et quelques autres paramètres, puis implémenter une valeur initiale à la variable d'optimisation. Ensuite, l'algorithme fait appel à la fonction *minimize* et la sortie de cette fonction (si la fonction *minimize* a convergé) est la valeur finale des variables d'optimisation. A partir de cette valeur finale, l'algorithme détermine toutes les grandeurs physiques qui nous intéressent, et les renvoye.

La réalité est un peu plus complexe, notamment pour des questions de temps de calculs. Le calcul de $R$ prend beaucoup de temps (0.5 secondes pour $n = 10$) alors même qu'il est fait des milliers de fois par appel de la fonction *minimize.* Il a donc fallu remédier à ce problème, sans quoi le programme a besoin de 'tourner' plusieurs heures pour obtenir un résultat dans les cas les plus simples.

## La linéarisation des flux radiatifs

Pour réduire le temps de calcul de $R$, une solution simple est la linéarisation : pour un $x_0$ donné, $R$ est calculé ainsi:

$$R(x) = R(x_0) + \nabla R(x_0).x \qquad \text{où } \nabla R(x_0) \text{ désigne le gradient de } R \text{ en } x_0$$

Si les calculs de $R(x_0)$ et $\nabla R(x_0)$ prennent un peu de temp, le calcul de $R(x)$ est quasiment instantané, et tant qu'il est relativement proche de $x_0$, il est une excellente approximation de la valeur exacte (à $10^{-15}$ près).

L'intérêt est clair : linéariser réduit par un facteur 1000 (voir beaucoup plus) le temps de résolution. Mais il y a un inconvénient : les fonctions dépendant de $R$ renvoient elles aussi des valeurs approchées, et non exactes. Et cela représente une difficulté de plus pour la convergence de la fonction *minimize* : même si la fonction converge et que le résultat semble réaliste, il se peut qu'il ne soit pas exact car calculé de manière approximative. Il est alors nécessaire relancer la fonction *minimize* à partir des derniers résultats obtenus afin de les affinés. Avant que les résultats ne soient corrects (ou du moins robustes), il faut parfois faire appel à *minimize* une vingtaine ou trentaine de fois. La démarche adoptée est détaillée dans le paragraphe sur les méthodes de résolution.

La question est dont de savoir à quelle fréquence il faut linéariser le calcul de $R$ afin de converger rapidement vers la solution exacte. Pour trouver un compromis entre convergence très lentement vers la solution exacte et convergnce rapideme vers une solution approchée, plusieurs fréquences de linéarisation ont été essayé :

- linéariser tous les K appels de la fonction de calcul des flux radiatifs
  mais cela n'est pas concluant pour K < 500 généralement..
- linéariser lorsque l'écart entre $x$ et $x_0$ dépasse un seuil donné.

cela fonctionne relativement bien, mais il s'avère que cela revient à peu près à la même chose que la proposition suivante

- linéariser à chaque nouvel appel de la fonction *minimize*

D'expérience, la solution la plus simple est finalement la dernière.

## Méthode de résolution

La fonction optimize ne trouve donc jamais la solution exacte en un seul appel, à cause de la linéarisation des flux radiatifs. Il n'y a pas de manière simple pour déterminer le nombre idéal d'appel, et plusieurs méthodes ont été imaginées pour trouver une solution satisfaisante. Ces méthodes – appelées arbitrairement de résolution pour ne pas les confondre avec la méthode d'optimisation de *minimize* – sont assez différentes les unes des autres, et dépendent de paramètres propres. Ces paramètres propres sont appelés ici « entrée », et sont modifiables par l'utilisateur, certaines combinaisons étant plus efficaces que d'autres.

### Méthode simple

Entrée : le nombre d'iteration (nmax)
Méthode : la fonction *minimize* est appelée n fois, avec comme valeur initiale le dernier résultat obtenu

### Méthode convergence

Entrée : le maximum d'itérations possible (nmax)
Méthode : la fonction *minimize* est appelée jusqu'à ce qu'elle converge, c'est-à-dire que la valeur de l'entropie trouvée est la même pendant 5 itérations successives. Si la fonction *minimize* ne converge pas, l'algorithme s'arrête et renvoie les résultats de l'itération nmax

### Méthode threshold

Entrée : la variation maximale tolérée de l'entropie (t), le maximum d'itérations possible (nmax)
Méthodes : la fonction *minimize* est appelée tant que la variation d'entropie et la variation de $x_i$ entre deux itérations est supérieure à t. En cas de non-convergence et non atteinte de ce seuil minimum de variation de valeur, l'algorithme renvoie les résultats de l'itération nmax

### Méthode maxiter variation

Entrée : la liste des maxiter à tester (l), le nombre d'itérations (n)
Méthode : pour toutes les valeurs de l, la fonction *minimize* est appelée tant qu'elle ne converge pas ou que le nombre d'itération est inférieur à n.

### Méthode value variation

Entrée : nombre d'itérations (n), la variable qui doit être changer (v), la valeur minimale et la valeur maximale que peut prendre cette variable (v min et v max), le nombre d'itérations par cycle (c)
Méthode : A chaque cycle d'itération (ie toutes les c itérations), la valeur d'une des variables d'optimisation (xi par exemple) est changée au hasard, la nouvelle valeur étant comprise entre 10**(v min) et 10**(v max). Si les résultats sont meilleurs que l'itération précédente, alors ils deviennent la valeur initiale de l'itération suivante. Sinon ce sont les derniers meilleurs résultats qui servent de valeur initiale. Cette méthode ne peut renvoyer des résultats intéressants que pour un très grand nombre d'itérations avec une fonction objectif relativement convexe, elle est peu intéressante de ce point de vue-là. Par contre elle permet de mieux comprendre l'évolution de l'algorithme d'optimisation, de sortir d'un minimum local et de voir la sensibilité de la fonction objectif à la variation d'une variable en particulier.

## Dernières améliorations pour une convergence plus robuste

## Les changements de variables

La fonction *minimize* est conçue pour que les valeurs des variables d'optimisation soient aux alentours de 1, à quelques puissances de 10 près. Pour faciliter la convergence de *minimize*, il peut être intéressant d'effectuer des changements de variables pour que les variables comme $x, m, F$ et $e$ soient dans les ordres de grandeurs recherchés.

Pour cela, deux changements de variables ont été fait :

- $F \rightarrow f \ : \ f = {F}/{\frac{S}{4}}$
  Ce changement de variable permet d'obtenir un ordre de grandeur de $F$ proche de la dizaine

- $E \rightarrow e \ : \ e = {E}/{\frac{S}{4}}$
  Pour être similaire à $F$
- $m \rightarrow z \ : \ z = \mathrm{c_a} + \mathrm{c_m} * \tanh(\frac{m}{c_{ref}})$
  Ce changement de variables un peu complexe a l'avantage de transformée la variable $m$ qui peut tendre vers l'infini en une variable bornée $z$.
  Les valeurs des coefficients $c_a, c_m$ et $c_{ref}$ peuvent beaucoup influencer les résultats car la fonction *minimize* semble peu sensible aux valeurs de $m$ (dans le sens où les résultats sont très proches à partir du moment où le profil de $m$ à une certaine forme, quelques soit son amplitude).
  $c_{ref}$ est calculé pour que, pour un pourcentage $p$ et une valeur de référence de mass $m_{ref}$, $p = \tanh(\frac{m_{ref}}{c_{ref}})$. Ce choix permet ainsi de mieux centrer le changement de variable sur les ordres de grandeurs des solutions.
  Pour les coefficiants $\mathrm{c_a} + \mathrm{c_m}$, leurs valeurs permet de rééquilibrer le poids de $m$ par rapport à $x$ : les conditions de terminaison de la fonction *minimize* ne sont pas bien connues, il semblerait que des valeurs de $m$ qui soit légèrement supérieur (de l'ordre de 2) par rapport aux valeurs de $x$ (de l'ordre de 1) accélère la convergence de *minimize.*
  D'autres changement de variable sur $m$ peuvent être effectués, un autre a été codé (mais est finalement un peu moins intéressant) :

$$z = \mathrm{c_a} + \mathrm{c_m} * (1 - \exp(\frac{m}{c_{ref}}))$$

## Test de 'feasibility'

Le problème d'optimisation peut ne pas être 'faisable'(traduction de l'anglais, donc peut-être impropre ?), ou bien il peut être difficile à résoudre parce que les valeurs faisables sont difficiles à trouver. Afin de vérifier la faisabilité d'un problème ou d'obtenir un point faisable, une possibilité est de résoudre le problème qui possède les mêmes contraintes, mais avec une fonction objective convexe très simple.

Le programme développé permet de faire le test de 'feasibility' et d'utiliser les valeurs obtenues ainsi comme les valeurs initiales du problème à résoudre. L'algorithme d'optimisation est parfois plus rapide ainsi, et ses résultats plus intéressants ou robustes.

Afin de pouvoir trouver plusieurs points faisables, avec différentes méthodes, le programme permet de choisir entre plusieurs fonctions objective convexes simple :

- une fonction constante (égale à zéro)
  → le problème de « feasibility » revient à déterminer les valeurs des variables d'optimisation respectant les contraintes les plus simple à trouver à partir des valeurs initiales

- une fonction linéaire égale à la somme des valeurs des variables d'optimisation
  → le problème de « feasibility » revient à déterminer les valeurs des variables d'optimisation respectant les contraintes dont la somme est maximale
- une fonction quadratique : $x \rightarrow \frac{1}{2}(x-a)^2$ où $a$ est une constante (à choisir en entrée de l'algorithme)
  → le problème de « feasibility » revient à déterminer les valeurs des variables d'optimisation respectant les contraintes dont les valeurs sont les plus proches possibles de $a$

  remarque : il y a l'ajout d'un signe - lorsque qu'il s'agit d'un problème de maximisation à résoudre avec la fonction *minimize*

### La résolution en 2 étapes

Comme cela donnait des résultats intéressant en C, la résolution en 2 étapes (d'abord avec les variables d'optimisation $x\&m$, puis avec $f$) a été codée en python. Néanmoins les résultats n'ont pas été concluant jusqu'à maintenant.

## Récapitulatif des choix de paramètres possibles pour le calcul numérique et de l'algorithme complet

### Choix possibles des paramètres

En prenant en compte tous les possibilités présentées, il y a tout un choix de paramètres à faire pour initialiser le programme :

- Paramètres de modélisation
  - le nombre de boîtes $n$
  - le modèle MEP (sans ou avec cycle de l'eau, à saturation ou non saturé..)
  - l'albedo et l'indice de profile

- Paramètres d'optimisation
  - les variables d'optimisation (en cohérence avec le choix du modèle)
  - [si $m$ fait partie des variables d'optimisation] la masse maximale qui ne peut être dépassée
  - l'expression de l'entropie en fonction des variables d'optimisation
  - la méthode d'optimisation
  - le maxiter

- Paramètre de résolution
  - la fréquence de linéarisation de $R$
  - la méthode de résolution et tous les paramètres liés
  - [optionnel] la valeur initiale des variables d'optimisation, qui peut être obtenue à partir du test de 'feasibility'

- Paramètre du changement de variables de $m$
  - [optionnel] les coefficients du changement de variable de $m$ (ou l'absence de changement de variable)

Le choix de paramètres varie en fonction de la version – simple, complète ou exhaustive – du programme (cf partie suivante).

## Récapitulatif de l'algorithme complet

Les différentes étapes sont donc :

- implémentation de tous les choix mentionnés plus hauts
- initialisation de toutes les valeurs avec les paramètres par défaut lorsque les choix n'ont pas été fait
- initialisation de toutes les données dépendant de $n$ et de l'albédo et de l'indice de profile (le profil de pression, les matrices auxiliaires de calcul.. ), des choix de modèles et de variables (l'expression de l'entropie et des contraintes), du choix de fréquence de linéarisation etc.. et création du dictionnaire qui contiendra les résultats
- application de la méthode de résolution itérative :
    - test pour vérifier qu'il y a encore besoin d'une itération (selon le protocole de la méthode de résolution)
    - linéarisation de la fonction de calcul de $R$
    - appel à la fonction *minimize*
    - [optionnel] affichage des résultats ($x$ et éventuellement $m$)
    - implémentation des résultats en tant que valeur initiale du prochain appel de la fonction *minimize*
- A la fin du protocole de la méthode de résolution, c'est-à-dire quand il y a eu convergence ou atteinte de maximum d'itérations possibles, les résultats sont enregistrés pour pouvoir être ensuite sauvegardés sous forme d'excel et/ou affichés sous forme de graphique.

# IV. Le programme python et les différentes versions

## Généralité sur le programme python :

### La nomenclature

Afin de simplifier la sauvegarde et le traitement des résultats d'un problème lié à un certain modèle, un nom est attribué à chaque modèle. Le nom est construit de la manière suivante :

(2 premières lettres de la modélisation des flux convectifs)_(constrainte)_(variable d'optimisation)_(2 premières lettres de la méthode d'optimisation

Les combinaisons possibles sont donc :

- ***'in' (***pour le modèle initial sans cycle de l'eau) ou ***'wc_sa'*** (pour le cycle de l'eau à saturation) ou ***'wc_ns'*** (pour le cycle de l'eau non saturé)
- si le modèle est celui initial, il faut ajout la contrainte ***'moist'*** ou ***'dry'*** ou ***'cpT'*** ou '***no'*** en fonction de la définition de l'énergie
- ***'x'*** ou ***'xm'*** ou ***'f'*** ou ***'xmf'*** ou ***'xmh'*** pour les variables d'optimisation
- ***'SL'*** (pour SLSQP) ou ***'tr'*** (pour trust-constr) pour la méthode d'optimisation

Par exemple, le modèle de Vincent prenant en compte l'humidité de l'air est le '***in_moist_f_SL'***.

### Les différents fichiers du programme

Le programme est composé de 8 fichiers :

- *main*
  ce fichier sert d'interface pour choisir les options et les paramètres du problème à résoudre
- *intermediate*
  ce fichier comporte quelques fonctions pour simplifier le fichier main, afin de transformer les options et paramètres choisis en données exploitables pour l'algorithme de résolution
- *mep_optimisation*
  ce fichier contient tout le code de résolution et d'optimisation
- *mep_physics*
  ce fichier contient tout le code lié à la physique du problème d'optimisation
- *physics*
  ce fichier contient les fonctions physics qui sont utiles aux 3 fichiers *profile_bis, radiativ* et *mep_physics*
- *profile_bis*
  le même fichier que profile du code de Vincent (simple changement de nom)
- *radiativ*
  le même fichier que radiativ du code de Vincent
- *plot_save*
  ce fichier contient les fonctions qui permettent la sauvegarde des résultats sous format *Excel* et de graphiques

## Les 3 différentes versions du code

### *version simple*

Cette version du code a été conçue pour être la plus simple possible, afin qu'elle soit facile à comprendre et qu'elle puisse servir de base pour un code plus complexe (un peu comme celle donnée par Vincent pour son article).

Les choix possibles avec cette version :

- le nombre de boîtes $n$,
- le modèle MEP (sans ou avec cycle de l'eau, à saturation ou non saturé..),
- l'albedo et l'indice de profile,
- les variables d'optimisation (en cohérence avec le choix du modèle),
- le nombre d'itérations et le seuil pour la méthode de résolution « threshold »,
- le maxiter,
- l'affichage des résultats, l'affichage et la sauvegarde des graphiques, la sauvegarde sous format excel.

Il n'y a pas de résolution comparative dans cette version.

### *version complète*

Cette version est celle qui contient à peu près toutes les options intéressantes, et qui peut être utilisée en « boîte noire » .

Les choix en plus par rapport à la version simple :

- la méthode d'optimisation,
- possibilité de comparer différents modèles,
- plus de méthode de résolution possibles,
- les différentes options de linéarisation de $R$,
- l'expression de la fonction de changement de variable $m \rightarrow z$ .

### *version exhaustive*

Cette version présente toutes les options qui ont été développées. Certaines étant peu utiles, mais elles peuvent avoir un intérêt bien précis lors du développement d'un nouveau modèle.

Les choix en plus par rapport à la version complète :

- l'expression de l'entropie quand celle-ci peut dépendre de plusieurs variables
- la résolution en 2 étapes
- option test de « feasibility »
- option sur la sauvegarde et l'affichage de l'évolution des solutions
- option sur la sauvegarde et l'affichage des valeurs initiales

# La version simple

## Le choix des paramètres avec main.py

### *Fonctionnement du choix des paramètres*

Pour choisir les valeurs des paramètres, il suffit de rentrer les valeurs dans le dictionnaire paramètres

Il est possible de commenter avec # un paramètre qui n'est pas utile.

### *Paramètres de modélisation*

- **'index of profile'** ***: entre 1 et 5,***
- **'albedo'** ***: 0.1 ou 0.6, en fonction du profile,***
- **'number of levels' :** ***entre 2 et 100,***
- **'physical model'**: ***'ini_moist'/'ini_no'/'ini_cpT'/ 'ini_dry'/ 'wc_sa'/ 'wc_ns'***

### *Paramètres de modélisation et résolution*

#### Linéarisation des flux radiatifs

Par défaut, les flux sont linéarisés à chaque itération de 'résolution'

#### Variables d'optimisation

- **'optimization variable' :** ***'x', 'xm', 'f', 'xmf', 'xmh'***

#### Méthode d'optimisation et maxiter

Par défaut, la méthode d'optimisation est SLSQP, et le maxiter est celui par défaut, c'est-à-dire 1000

#### Méthode de résolution

Par défaut, la méthode de résolution est *'threshold', avec 'variation threshold'* = $10^{-8}$ et '*max number of iterations' =30*

#### Initialisation des variables d'optimisation [optionnel]

- **'optimization variable to initialize' :** ***'m'/'x'/'xm'/'f'/'h',***
- **'initial value':** ***(float or array of the right length)***

### *Paramètres d'affichage des résultats*

Il est possible de voir l'évolution de la résolution par l'algorithme avec le paramètre :

- ***'save to excel option' : True/False***

### *Paramètres de sauvegarde sous format excel*

Il est possible de sauvegarder certaines données dans un fichier Excel. Attention, pour que l'Excel soit sauvegardé, il faut créer en amont dans le dossier contenant les fichiers de code python un nouveau dossier appelé 'new results'. Là encore, pour automatiser au maximum le programme, un choix d'option est possible :

- ***'save to excel option' : True/False***

### *Paramètres de l'affichage et de la sauvegarde des graphiques*

Il est possible d'afficher les graphiques présentant des grandeurs physiques (parmi $T, M, F, E, h, q, L$) par rapport à la pression $p$. Attention, pour que les graphiques soient sauvegardés, il faut créer en amont

dans le dossier contenant les fichiers de code python un nouveau dossier appelé ‘new results’. Là encore, pour automatiser au maximum le programme, un choix d’option est possible :

- ***'plotting the graphics option' : True/False,***
  option d’affichage et de sauvegarde des graphiques (si False, les graphiques ne seront ni affichés, ni enregistrés)
- ***'list value to plot': (list)*** *exemple ['E','T','M', 'L']*
  liste des grandeurs physiques à afficher, parmi ['E','T','F','q','M', 'L']

## L’ajout d’un modèle ou d’une contrainte

Pour ajouter un modèle, il faut adapter le code des fonctions déterminant les valeurs initiales, les contraintes, les fonctions objectives, le calcul des variables physique, et toutes les fonctions de mep_physique en rapport.

Si une contrainte est ajoutée, il faut modifier le dictionnaire de plot_and_save pour que la contrainte puisse entre enregistrée sous le format excel

# La version complète

## Le choix des paramètres avec main.py

### *Fonctionnement du choix des paramètres*

Pour choisir les valeurs des paramètres, il suffit de rentrer les valeurs dans le dictionnaire paramètres

Il est possible de commenter avec # un paramètre qui n'est pas utile.

### *Paramètres de modélisation*

- **'index of profile'** ***: entre 1 et 5,***
- **'albedo'** ***: 0.1 ou 0.6, en fonction du profile,***
- **'number of levels' :** ***entre 2 et 100,***
- **'physical model'**: ***'ini_moist'/'ini_no'/'ini_cpT'/ 'ini_dry'/ 'wc_sa'/ 'wc_ns'***

### *Paramètres de modélisation et résolution*

#### Linéarisation des flux radiatifs

- **'linearisation of radiativ flux' :** ***'once per resolution iteration'/'no linearisation'/'every K call' /'variation of x'***

  si *'every K call'*:
  - ***'K': (int)***

  si *'variation of x'*:
  - **'linearisation variation threshold':** ***(float)***

#### Variables d'optimisation

- **'optimization variable' :** ***'x', 'xm', 'f','mf', 'xmf', 'xmh'***

  si *'m'* appartient aux variables d'optimisation :
  - **'maximal mass' :** ***(float)***
  - **'function of the variable change':** ***'id'/'simple multiplication'/'tanh'/'1-exp'***

    si *'tanh'* ou *'1-exp'*:
    - **'coefficiant multiplication':** ***(float)***
    - **'coefficiant addition':** ***(float)***
    - **'mass reference':** ***(float)***
    - **'percent reference' :** ***(float)***

    si 'multiplication':
    - **'coefficiant multiplication':** ***(float)***

#### Méthode d'optimisation et maxiter

- **'optimization method' :** ***'SLSQP'/'trust-constr'***
- **'maxiter of the minimize function' :** ***(float)***

#### Méthode de résolution

Par défaut, la méthode de résolution est *'threshold', mais il faut encore ajouter :*

- **'variation threshold'** ***: (float)***
- **'max number of iterations' :** ***(int)***

#### Initialisation des variables d'optimisation [optionnel]

- **'optimization variable to initialize' :** ***'m'/'x'/'xm'/'f'/'h'***

- **'initial value':** ***(float or array of the right length)***

Test de 'feasibility' [optionnel]:

- [optionnel] **'feasibility':** ***'resolution'/'test and resolution'/'test'***
  si *'test and resolution' ou 'test'*, par défaut, la *'feasibility objective function'* est *'null'*
  si ce paramètres ne sont pas précisé, par défaut la valeur sera 'resolution'

### *Paramètres d'affichage des résultats*

Il est possible de voir l'évolution de la résolution par l'algorithme avec le paramètre :

- ***'save to excel option' : True/False***

### *Paramètres de sauvegarde sous format excel*

Il est possible de sauvegarder certaines données dans un fichier Excel. Attention, pour que l'Excel soit sauvegardé, il faut créer en amont dans le dossier contenant les fichiers de code python un nouveau dossier appelé 'new results'. Là encore, pour automatiser au maximum le programme, un choix d'option est possible :

- ***'save to excel option' : True/False***
  option de sauvegarde des données dans un fichier Excel
- ***'initial value save option' : True/False,***
  option de sauvegarde des valeurs initiales (intéressant si l'option feasibility test a été choisie)
- ***'evolution save option' : True/False***,
  option de sauvegarde de l'évolution : toutes les K itérations, les résultats de la fonction *minimize* sont enregistrés pour être ensuite sauvegardés dans l'Excel
- si 'evolution save option' = True: ***'nb iteration for saving' : (int)***,
  choix du nombre d'itérations du cycle de sauvegarde (ie K). Attention, il d'agit d'un nombre entier !

### *Paramètres de l'affichage et de la sauvegarde des graphiques*

Il est possible d'afficher les graphiques présentant des grandeurs physiques (parmi $T, M, F, E, h, q, L$) par rapport à la pression $p$. Attention, pour que les graphiques soient sauvegardés, il faut créer en amont dans le dossier contenant les fichiers de code python un nouveau dossier appelé 'new results'. Là encore, pour automatiser au maximum le programme, un choix d'option est possible :

- ***'plotting the graphics option' : True/False,***
  option d'affichage et de sauvegarde des graphiques (si False, les graphiques ne seront ni affichés, ni enregistrés)
- ***'list value to plot': (list)*** *exemple ['E','T','M', 'L']*
  liste des grandeurs physiques à afficher, parmi ['E','T','F','q','M', 'L']
- ***'evolution plot option': True/False,***
  option d'affichage sur les graphiques de l'évolution
- ***'initial value plot option': True/False,***
  option d'affichage sur les graphiques de la valeur initiale

# La version exhaustive

## Le choix des paramètres avec main.py

### *Fonctionnement du choix des paramètres*

Pour choisir les valeurs des paramètres, il suffit de rentrer les valeurs dans le dictionnaire paramètres

Il est possible de commenter avec # un paramètre qui n'est pas utile.

### *Paramètres de modélisation*

- **'index of profile'** ***: entre 1 et 5,***
- **'albedo'** ***: 0.1 ou 0.6, en fonction du profile,***
- **'number of levels' :** ***entre 2 et 100,***
- **'physical model'**: ***'ini_moist'/'ini_no'/'ini_cpT'/ 'ini_dry'/ 'wc_sa'/ 'wc_ns'***

### *Paramètres de modélisation et résolution*

#### Linéarisation des flux radiatifs

- **'linearisation of radiativ flux' :** ***'once per resolution iteration'/'no linearisation'/'every K call' /'variation of x'***

  si *'every K call'*:
  - ***'K': (int)***

  si *'variation of x'*:
  - **'linearisation variation threshold':** ***(float)***

#### Variables d'optimisation

- **'optimization variable' :** ***'x', 'xm', 'f', 'xmf', 'xmh'***

  si *'m'* appartient aux variables d'optimisation :
  - **'maximal mass' :** ***(float)***
  - **'function m to z' :** ***'id'/'multiplication'/'tanh'/'1-exp'***

    si *'tanh'* ou *'1-exp'*:
    - **'coefficiant multiplication':** ***(float)***
    - **'coefficiant addition':** ***(float)***
    - **'mass reference':** ***(float)***
    - **'percent reference' :** ***(float)***

    si 'multiplication':
    - **'coefficiant multiplication':** ***(float)***

#### Méthode d'optimisation et maxiter

- **'optimization method' :** ***'SLSQP'/'trust-constr'***
- **'maxiter of the minimize function' :** ***(float)***

#### Méthode de résolution

- **'resolution method' :** ***'threshold'/'simple'/'convergence'/'maxiter'/'value variation'***

  si 'simple :
  - **'max number of iterations' :** ***(int)***

  si 'threshold' :

- **'variation threshold'** ***: (float)***
- **'max number of iterations' : *(int)***

Initialisation des variables d'optimisation [optionnel]

- **'optimization variable to initialize' : *'m'/'x'/'xm'/'f'/'h'***
- **'initial value': *(float or array of the right length)***

Expression de l'entropie en fonction des variables [optionnel]

- [optionnel]**'entropy variable' : 'x'/'xm'/'xf'**

Résolution en 2 étapes [optionnel]:

- [optionnel] **'double step': *True/False***
  si *True* :
  - **'nb double step iteration' : *(entiers)*,**

  si ces paramètres ne sont pas précisés, par défaut la résolution sera en 1 étape, classiquement

Test de 'feasibility' [optionnel]:

- [optionnel] **'feasibility': *'resolution'/'test and resolution'/'test'***
  si *'test and resolution' ou 'test'*:
  - **'feasibility objective function': *'null'/'maximal sum'/'minimal sum'/'sum equal'***

  si 'sum equal':
  - **'to equalize'** ***: m/x/f***
  - **'value to be equal to': (float)**

  si ce paramètres ne sont pas précisé, par défaut la valeur sera 'resolution'

*Paramètres d'affichage des résultats*

Il est possible de voir l'évolution de la résolution par l'algorithme avec le paramètre :

- ***'save to excel option' : True/False***

*Paramètres de sauvegarde sous format excel*

Il est possible de sauvegarder certaines données dans un fichier Excel. Attention, pour que l'Excel soit sauvegardé, il faut créer en amont dans le dossier contenant les fichiers de code python un nouveau dossier appelé 'new results'. Là encore, pour automatiser au maximum le programme, un choix d'option est possible :

- ***'save to excel option' : True/False***
  option de sauvegarde des données dans un fichier Excel
- ***'initial value save option' : True/False,***
  option de sauvegarde des valeurs initiales (intéressant si l'option feasibility test a été choisie)
- ***'evolution save option' : True/False***,
  option de sauvegarde de l'évolution : toutes les K itérations, les résultats de la fonction *minimize* sont enregistrés pour être ensuite sauvegardés dans l'Excel
- si 'evolution save option' = True: ***'nb iteration for saving' : …(int)***,
  choix du nombre d'itérations du cycle de sauvegarde (ie K). Attention, il d'agit d'un nombre entier !

### *Paramètres de l'affichage et de la sauvegarde des graphiques*

Il est possible d'afficher les graphiques présentant des grandeurs physiques (parmi $T, M, F, E, h, q, L$) par rapport à la pression $p$. Attention, pour que les graphiques soient sauvegardés, il faut créer en amont dans le dossier contenant les fichiers de code python un nouveau dossier appelé 'new results'. Là encore, pour automatiser au maximum le programme, un choix d'option est possible :

- ***'plotting the graphics option' : True/False,***
  option d'affichage et de sauvegarde des graphiques (si False, les graphiques ne seront ni affichés, ni enregistrés)
- ***'list value to plot': (list)*** *exemple ['E','T','M', 'L']*
  liste des grandeurs physiques à afficher, parmi ['E','T','F','q','M', 'L']
- ***'evolution plot option': True/False,***
  option d'affichage sur les graphiques de l'évolution
- ***'initial value plot option': True/False,***
  option d'affichage sur les graphiques de la valeur initiale